

Concurrency in Money Withdrawal



In a Spring Boot REST API, if multiple users try to withdraw money at the same time, how will you handle it?

In a **Spring Boot REST API**, handling **simultaneous money withdrawal requests** is a classic **concurrency problem**. If multiple users try to withdraw at the same time, we must ensure **data consistency** so that the account balance never becomes incorrect.

This problem is usually solved using **database transactions + locking mechanisms**.

1 Problem Without Concurrency Control

Suppose account balance = ₹1000

Two requests come at the same time:

Request	Action
User A withdraws ₹800	
User B withdraws ₹500	

Without proper control:

1. Thread A reads balance = 1000
2. Thread B reads balance = 1000
3. Thread A updates balance → 200
4. Thread B updates balance → 500

Final balance = ₹500 ❌ (Incorrect)

Correct result should be **only one transaction succeed**.

2 Solution 1 — Database Transaction + Row Lock (Best Approach)

Use **transaction + pessimistic locking**.

Java

```
@Transactional
public void withdraw(Long accountId, double amount) {
    Account account = accountRepository.findByIdForUpdate(accountId);

    if(account.getBalance() < amount){
        throw new RuntimeException("Insufficient balance");
    }
}
```

```

    }

    account.setBalance(account.getBalance() - amount);
    accountRepository.save(account);
}

```

Repository:

Java

```

@Lock(LockModeType.PESSIMISTIC_WRITE)
@Query("SELECT a FROM Account a WHERE a.id = :id")
Account findByIdForUpdate(Long id);

```

How it works:

1. First request locks the row
2. Second request **waits**
3. First transaction completes
4. Second transaction sees updated balance

✓ Prevents race condition.

3 Solution 2 — Optimistic Locking (Most Common in Microservices)

Use **version field**.

Java

```

@Entity
public class Account {

    @Id
    private Long id;

    private double balance;

    @Version
    private Integer version;
}

```

Flow:

1. Read account with version = 1
2. Update balance
3. Database checks version

If another update happened:

OptimisticLockException

Then retry transaction.

✓ Very scalable.

4 Solution 3 — Distributed Lock (Microservices)

If system runs on multiple servers:

Use **Redis distributed lock**

Example:

```
SET lock_account_101 NX EX 10
```

Only one service instance can update account.

Libraries:

- Redisson
- Redis Lock

5 Solution 4 — Synchronized Block (Not Recommended)

Java

```
synchronized(accountId.toString().intern()) {
    withdrawLogic();
}
```

- ✗ Works only inside **single JVM**
- ✗ Not suitable for microservices.

6 Best Production Architecture

Typical banking system uses:

```
Spring Boot
  ↓
@Transactional
  ↓
Optimistic Locking
  ↓
Database (PostgreSQL / MySQL)
```

Sometimes combined with:

- Event queue
- Distributed locks

★ Simple Interview Answer

If asked in interview:

I would handle concurrent withdrawals using **database transactions with optimistic or pessimistic locking**.

In Spring Boot, we can use **@Transactional** with **@Version for optimistic locking** or **PESSIMISTIC_WRITE lock** to ensure that only one transaction updates the account balance at a time, preventing race conditions.

✓ If you want, I can also show **how Paytm / banking systems handle millions of withdrawals safely** (real industry architecture). It's a **very common system design interview question**.



what is optimistic and pessimistic lock

In **concurrent systems** (like a **Spring Boot REST API** or databases), **locking** is used to prevent **race conditions** when multiple users try to modify the same data.

Two common strategies are:

- 1 **Optimistic Locking**
- 2 **Pessimistic Locking**

1 Optimistic Locking

Idea:

Assume that **conflicts are rare**, so don't lock the data while reading.

Instead, check **before updating** whether someone else has modified the data.

How it works

A **version number** (or timestamp) is stored with the record.

Example table:

id	balance	version
1	1000	1

Steps

1. Transaction A reads account → version = 1
2. Transaction B reads account → version = 1
3. A updates balance → version becomes **2**
4. B tries to update → version mismatch ❌

Database throws:

`OptimisticLockException`

Then B must **retry**.

Example in JPA

Java

```
@Entity
public class Account {

    @Id
    private Long id;

    private double balance;

    @Version
    private Integer version;
}
```

Spring automatically checks the version before updating.

Advantages

- ✓ High performance
- ✓ No waiting/locking
- ✓ Best for **high-read systems**

Disadvantages

- ✗ Transaction may fail and must retry

2 Pessimistic Locking

Idea:

Assume that **conflicts will happen**, so lock the data **before reading/updating**.

Only one transaction can access the record at a time.

Steps

1. Transaction A reads account **with lock**
2. Transaction B tries to read → **must wait**
3. A finishes transaction
4. B continues

Example in JPA

Java

```
@Lock(LockModeType.PESSIMISTIC_WRITE)
@Query("SELECT a FROM Account a WHERE a.id = :id")
Account findByIdForUpdate(Long id);
```

Here the database applies a **row lock**.

Advantages

- ✓ Very safe
- ✓ No update conflicts

Disadvantages

- ✗ Threads must wait
- ✗ Can reduce performance under heavy load

3 Simple Real-World Analogy

Optimistic Lock

Library book system:

- Anyone can take the book record.
- When returning, system checks if someone already modified the record.

If yes → retry.

Pessimistic Lock

ATM machine:

- When one person is withdrawing money,
- the ATM **locks the account temporarily**.

Others must wait.

4 Quick Interview Comparison

Feature	Optimistic Lock	Pessimistic Lock
Lock time	No lock during read	Lock immediately
Performance	High	Lower
Conflict handling	Retry transaction	Wait
Best use	Read-heavy systems	High-conflict systems

5 What is Mostly Used in Spring Boot?

Most systems prefer **Optimistic Locking** because it **scales better in microservices**.

✓ One interview tip:

If interviewer asks about concurrency in **money withdrawal**, say:

"I would use **optimistic locking with a version field and @Transactional**, so that if two transactions try to update the same account, one will fail and retry."

If you want, I can also explain a **tricky interview question**:

Why banks sometimes avoid optimistic locking for balance updates (very interesting concept).



Solution 3 — Distributed Lock (Microservices)

In **microservices**, multiple instances of your **Spring Boot** application may run on different servers. So **Java synchronized or local locks will not work**, because they only lock **inside one JVM**.

To control concurrency across multiple servers, we use a **Distributed Lock**.

A very common tool used is **Redis**.

1 What is a Distributed Lock?

A **distributed lock** ensures that **only one service instance across the entire system** can access a critical resource at a time.

Example:

Multiple users try to withdraw from the same account.

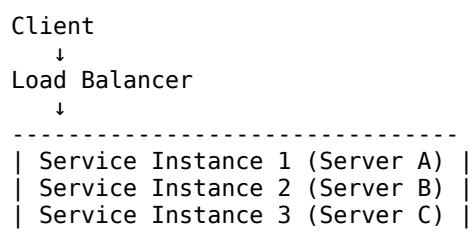
Service Instance 1 → withdraw request
Service Instance 2 → withdraw request
Service Instance 3 → withdraw request

Distributed lock ensures:

Only ONE instance processes withdrawal
Others wait or fail

2 Why We Need It in Microservices

Suppose your service is deployed like this:



If two requests hit different servers:

User A → Server A
User B → Server B

Local locks won't coordinate between servers.

So we use a **central lock system** like Redis.

3 How Redis Distributed Lock Works

Redis provides an atomic command:

```
SET lock_account_101 true NX EX 10
```

Meaning:

Parameter	Meaning
NX	Set only if key does not exist
EX 10	Expire after 10 seconds

Flow:

1 Service tries to acquire lock

```
SET lock_account_101 true NX EX 10
```

2 If success → lock acquired → process withdrawal

3 If failure → another request already holds lock

4 After completion → release lock

```
DEL lock_account_101
```

4 Example in Spring Boot (Using Redis)

Using **Redisson** library:

Java

```
RLock lock = redissonClient.getLock("lock_account_" + accountId);

try {
    if(lock.tryLock(5,10, TimeUnit.SECONDS)) {
        Account account = accountRepository.findById(accountId);

        if(account.getBalance() < amount){
            throw new RuntimeException("Insufficient balance");
        }

        account.setBalance(account.getBalance() - amount);
        accountRepository.save(account);
    }
} finally {
    lock.unlock();
}
```

5 What Happens Internally

Request 1 → acquire Redis lock → SUCCESS

Request 2 → acquire Redis lock → FAIL → wait/retry

Request 1 → completes → release lock

Request 2 → now acquires lock

This ensures **data consistency across servers**.

6 Real Production Use Cases

Distributed locks are used in:

- Payment systems
- Order processing
- Inventory management
- Ticket booking

Example:

BookMyShow seat booking
Flipkart inventory update
Bank withdrawal

7 Interview Summary

A strong answer:

In a microservices architecture with multiple service instances, I would use a **distributed lock using Redis**.

Each request tries to acquire a lock for the account ID before performing the withdrawal. If the lock is acquired, the transaction proceeds; otherwise, the request waits or retries. This ensures only one instance updates the account balance at a time.

✓ Pro Tip (Interview level):

Many systems combine:

Distributed Lock (Redis)
+
Database Transaction
+
Optimistic Locking

This gives **very strong consistency**.

If you want, I can also show **how companies like Amazon avoid distributed locks completely using another pattern** (very interesting system design concept).



Database Transaction

A **Database Transaction** is a group of database operations that are executed **as a single unit of work**. Either **all operations succeed**, or **none of them happen**.

This is critical in systems like a **Spring Boot REST API** when performing operations like **money withdrawal, payments, or order placement**.

1 Simple Example (Money Withdrawal)

Suppose a withdrawal process has two steps:

1. Deduct money from account
2. Save transaction record

Without a transaction:

Step 1: Deduct ₹500 from account

Step 2: Server crashes before saving transaction

Result:

- Money deducted ❌
- No transaction record ❌

Data becomes **inconsistent**.

With a **database transaction**, both steps happen together.

```

Start Transaction
  ↓
Deduct money
Save transaction record
  ↓
Commit Transaction
  
```

If something fails:

```

Start Transaction
  ↓
Deduct money
Error occurs
  ↓
Rollback
  
```

Database returns to **previous state**.

2 ACID Properties of Transactions

Every database transaction follows **ACID**.

A — Atomicity

All operations succeed **or none**.

Example:

```

Debit ₹500
Credit ₹500
  
```

If credit fails → debit also undone.

C — Consistency

Database always remains **valid**.

Example:

Balance cannot become negative

I — Isolation

Multiple transactions **do not interfere** with each other.

Example:

Two users withdrawing money simultaneously.

D — Durability

Once transaction commits, data is **permanently stored**.

Even if server crashes.

3 Transaction in Spring Boot

In **Spring Boot**, transactions are usually handled using `@Transactional`.

Example:

Java

```
@Service
public class AccountService {

    @Transactional
    public void withdraw(Long accountId, double amount) {

        Account account = accountRepository.findById(accountId).get();

        if(account.getBalance() < amount){
            throw new RuntimeException("Insufficient balance");
        }

        account.setBalance(account.getBalance() - amount);
        accountRepository.save(account);
    }
}
```

What @Transactional does

Spring automatically:

1. Starts transaction
 2. Executes method
 3. If success → **commit**
 4. If exception → **rollback**
-

4 Transaction Flow Internally

Client Request



```

Spring Boot Service
    ↓
@Transactional starts
    ↓
Database operations
    ↓
Success → COMMIT
Failure → ROLLBACK

```

5 Why Transactions Are Important for Concurrency

If multiple users withdraw money at the same time:

Transactions ensure:

- **Isolation**
- **Data consistency**

Often combined with:

- **Optimistic Locking**
- **Pessimistic Locking**

6 Interview-Level Answer

If asked "**What is a database transaction?**"

You can say:

A database transaction is a sequence of operations executed as a single unit of work. It follows ACID properties—Atomicity, Consistency, Isolation, and Durability—to ensure data integrity. In Spring Boot, transactions are typically managed using the `@Transactional` annotation, which ensures that all database operations inside the method either commit successfully or roll back if an error occurs.

✓ If you want, I can also explain **how @Transactional actually works internally using Spring AOP and proxies** (this is a **very common advanced Spring interview question**).